

Research Engineer — Take-Home Assignment

Building a Large Behavior Model on Twin-2K-500

Welcome, and thanks for taking the time. This exercise asks you to design (and partially prototype) an LLM-based **behavior model** — a system that simulates how individual people answer questions — using

a real, public dataset of 2,000+ people. We care far more about **how you think** than about how much you finish. A focused, well-reasoned partial submission beats a sprawling, hand-wavy complete one.

Background

LLM-based "digital twins" use a language model to emulate an individual's behavior: given what a person has said before, predict how they'd respond to **new** questions. This has real uses in social science, market research, and product/experiment design — but it's only credible if it's validated against real human ground truth.

The **Twin-2K-500** dataset gives you exactly that ground truth. It surveyed a representative sample of **N = 2,058 US adults** across **four waves** (~500 questions total: demographics, psychological scales, economic preferences, cognitive tasks, behavioral-economics experiments, and a pricing survey). Crucially, **wave 4 re-administers earlier questions**, which lets you (a) train on a person's earlier answers and predict their later ones, and (b) measure how consistent humans are with

themselves over time — a built-in performance ceiling.

- Dataset: <<https://huggingface.co/datasets/LLM-Digital-Twin/Twin-2K-500>> (license: CC BY 4.0)
- Docs: <<https://digital-twin-simulation-version2.readthedocs.io/en/latest/index.html>>
- Reference repo: <<https://github.com/tianyipeng-lab/Digital-Twin-Simulation>>
- Paper: Toubia et al., 2025 — <<https://arxiv.org/abs/2505.17479>>

You do **not** need to read the whole paper. Skim what you need.

Your task

Design a credible plan to build a **Large Behavior Model** on this dataset, back it with real exploration of the data, and (bonus) prototype a tiny piece of it. Frame everything around the dataset's natural prediction task:

***Given a person's answers from waves 1–3 (their "persona"), predict how that same person answers*

*the held-out wave-4 questions** — and evaluate against both the real wave-4 answers and the human*

test–retest ceiling.

This is a design-and-reasoning exercise with a hands-on component. We are **not** expecting a trained, production-quality model.

Deliverables

Submit a **public GitHub repository** containing the following. Each item lists what we're looking for; depth and clarity matter more than length.

1. Data exploration report

A short report (markdown/notebook/PDF in the repo) showing you actually opened the data. We'd expect

things like: dataset structure and the two HF subsets; question types and how they'd be scored; distributions and representativeness; **the test–retest reliability of humans** (your most important finding); and the dataset's biases/limitations. Include the code or notebook that produced it.

2. Concrete plan to build the model (with technical detail)

How you would actually build the behavior model. Be specific enough that another engineer could start

from it: problem framing, data pipeline, the modeling approach(es) you'd try and **why** (e.g. prompting vs. retrieval vs. fine-tuning vs. preference tuning), how you'd handle long personas and the context window, training details (base model, method, objective, key hyperparameters), and the main risks with

mitigations. Trade-offs and clear reasoning beat buzzwords.

3. Evaluation strategy

How you'd know the model is good. We're especially interested in: which metrics for which question

types; what you compare against (what's the **ceiling**? what's the trivial **baseline** to beat?); the train/test protocol; and how you'd **prevent data leakage** (this dataset has a specific trap — find it). State explicit acceptance criteria.

4. Business applications

What kinds of products or organizations could use this model's capabilities, and how. Be concrete about the job it does and — equally important — the **guardrails and what it should *not* be used for**.

5. Long-run maintenance

How you'd keep this model trustworthy over months/years: monitoring, drift, retraining triggers, versioning, governance/ethics. Behavior changes over time — how do you keep up?

6. (Bonus) Proof-of-concept training code

A small, runnable POC that fine-tunes a **model with fewer than 0.5B parameters** on a slice of the task, with an eval that reports a metric against a baseline. It does not need to perform well — it needs to demonstrate that your end-to-end loop (data → train → eval) actually works, and that you can reason about results from a tiny model. Include clear run instructions.

What we evaluate

- **Data sense** — did you genuinely understand the dataset, including its leakage trap and the test–retest ceiling?
- **Technical judgment** — are your modeling and evaluation choices sound, specific, and justified?
- **Rigor on evaluation** — strong candidates obsess over *what to compare against*, not just metrics.
- **Communication** — clear, honest writing; trade-offs stated; limitations acknowledged.
- **Code quality** (if you do the bonus) — clean, runnable, reproducible.
- **Honesty** — we value "I didn't have time to verify X" over overclaiming.

There is no single right answer. We're hiring for how you reason about an open-ended, real problem.

Logistics

- **Compute**: to train your POC model, take advantage of the free credits you get on first sign-up with **Google Colab** or **RunPod** — the bonus is sized to run comfortably within them. You may also

use the dataset's pre-computed LLM simulation results as baselines instead of paying for API calls.

- **Tools:** any language/framework; use of AI coding assistants is fine — we may ask you to walk through your reasoning.

- **Submission:** send a link to your **public GitHub repo** with a top-level README that orients us and explains how to run any code. A short (optional) note on what you'd do with more time is welcome.

Questions about scope? Reach out — knowing when to ask is part of the job.

Good luck, and have fun with it.